BlimBlom – คู่มือ Antigravity + ไฟล์เสียง + Prompt สั่งงาน

ส่วน A: หาไฟล์เสียง · ส่วน B: ติดตั้งและตั้งค่า Antigravity · ส่วน C: prompt สำเร็จรูปสำหรับสั่ง agent · ใช้คู่กับคู่มือ Cloud Run และคู่มือเครื่องมืออื่น ๆ

> **แก้ไขจากคำแนะนำก่อนหน้า:** โค้ดตัวอย่างปุ่ม Simulate ที่เคยให้ไป (`String.fromCharCode(...new Uint8Array(buffer))`) จะ error กับไฟล์เสียงขนาดหลายร้อย KB ซึ่งคือขนาดคลิปที่เราใช้ ห้ามใช้โค้ดนั้น ให้ใช้ prompt C2 ในคู่มือนี้แทน ซึ่งสั่งให้ใช้วิธีที่ปลอดภัย (`FileReader.readAsDataURL`)

ส่วน A – ไฟล์เสียง

A1. เสียงเลื่อยยนต์ เครื่องยนต์ และเสียงป่า: ESC-50 (GitHub)

- Repo: **https://github.com/karolpiczak/ESC-50**
- คลิปละ 5 วินาที เป็นไฟล์ WAV (mono, 44.1 kHz) ขนาดประมาณ 430 KB
- สัญญาอนุญาต: CC BY-NC 3.0 (ใช้เพื่อการแข่งขันที่ไม่ใช่การค้าได้) **ต้องใส่เครดิตในสไลด์**: "Sound clips from ESC-50 (Piczak, 2015), CC BY-NC 3.0"
- ไม่ต้องโหลดไฟล์ zip ทั้งก้อน (ประมาณ 600 MB) ใช้สคริปต์ `get_sounds.sh` ที่ส่งมาด้วย ดึงเฉพาะคลิปที่ต้องใช้ (ทดสอบแล้ว ได้ 21 ไฟล์)

```
# Cloud Shell, Mac Terminal หรือ Git Bash
bash get_sounds.sh

# Windows PowerShell
powershell -ExecutionPolicy Bypass -File get_sounds.ps1
```

ไฟล์จะไปอยู่ในโฟลเดอร์ `audio/` ชื่อเช่น `chainsaw_1.wav` แต่ละประเภทใช้แทนเหตุการณ์ดังนี้:

หมวดใน ESC-50	ใช้เป็นเหตุการณ์ BlimBlom
chainsaw, hand_saw	chainsaw (ลักลอบตัดไม้)
engine	vehicle (รถเข้าป่า)
crickets, chirping_birds, rain, insects	normal (เสียงป่าปกติ ใช้ทดสอบว่าระบบไม่แจ้งเตือนมั่ว)

A2. เสียงช้าง (ไม่มีใน ESC-50)

แหล่ง	ลิงก์	หมายเหตุ
Freesound (แนะนำ)	https://freesound.org/search/?q=elephant+trumpet https://freesound.org/search/?q=elephant+rumble	ต้องสมัครบัญชีฟรีก่อนดาวน์โหลด เลือกไฟล์ที่เป็น CC0 หรือ CC BY แล้วจดชื่อผู้อัปโหลดไว้ใส่เครดิต
Dryad: African elephant vocalizations (Samburu)	https://datadryad.org/dataset/doi:10.5061/dryad.xd2547dz3	โหลดเฉพาะ `clips.zip` (ประมาณ 120 MB) เป็นเสียง rumble ส่วนใหญ่ อ่าน README และเงื่อนไขการใช้ก่อน
Elephant Listening Project (Cornell)	https://www.elephantlisteningproject.org	ใช้ฟังเป็นตัวอย่างอ้างอิง ไม่ใช่แหล่งดาวน์โหลดหลัก

- **เลือกเสียงแตร (trumpet) เป็นหลัก** เพราะชัดและจำง่าย เสียง rumble ความถี่ต่ำมาก Gemini อาจแยกไม่ออก ให้ลองใน AI Studio ก่อนใช้จริง
- ตัดให้ยาวไม่เกิน 10 วินาที (ใช้ Audacity ฟรี หรือเว็บ mp3cut.net) แล้วตั้งชื่อ `elephant_1.mp3`, `elephant_2.mp3`
- เสียงที่หาได้ส่วนใหญ่เป็นช้างแอฟริกา สำหรับเดโมใช้ได้ แต่ถ้ากรรมการถามให้ตอบตรง ๆ ว่าเวอร์ชันจริงจะใช้เสียงช้างเอเชียจากพื้นที่จริง

A3. วางไฟล์ในโปรเจกต์

```
public/
  audio/
    elephant_1.mp3     -> S01 Kui Buri North Gate
    elephant_2.mp3     -> S02 Ban Rai Farm Edge
    chainsaw_1.wav     -> S03 Deep Forest East
    crickets_1.wav     -> S04 Stream Bank
```

ไฟล์ .wav ต้องส่ง `mime_type: "audio/wav"` ส่วน .mp3 ส่ง `"audio/mp3"` (prompt C2 สั่งให้ agent ทำให้อัตโนมัติ) ไฟล์ WAV 5 วินาทีมีขนาดไม่เกินที่ Cloud Run รับได้

ส่วน B – ติดตั้งและตั้งค่า Antigravity

Antigravity อัปเดตบ่อย ชื่อปุ่มอาจต่างจากนี้เล็กน้อย ดูคู่มือทางการที่ antigravity.google/docs ประกอบ

B1. ติดตั้ง

1. ดาวน์โหลดจาก **https://antigravity.google/download** (Windows, macOS, Linux บางรุ่น)
2. ติดตั้ง แล้วเลือก **Continue with Google** ล็อกอินด้วย Gmail ส่วนตัว
3. เลือก Theme, ข้าม plugin ได้, ยอมรับเงื่อนไข → **Finish**
4. ควรตั้ง Chrome เป็นเบราว์เซอร์หลัก เพราะคำสั่ง `/browser` ใช้ Chrome ทดสอบหน้าเว็บ

B2. ติดตั้งเครื่องมือประกอบบนเครื่อง (ทำครั้งเดียว)

- **Node.js LTS** จาก nodejs.org
- **Git** จาก git-scm.com (Windows จะได้ Git Bash มาด้วย ใช้รัน `get_sounds.sh` ได้)
- **Firebase CLI**: เปิด Terminal แล้วรัน `npm install -g firebase-tools` แล้ว `firebase login`
- Windows ถ้ารัน `firebase` แล้วขึ้นว่า running scripts is disabled ให้รัน `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` ใน PowerShell ครั้งเดียว

B3. เปิดโค้ดเป็น Project

1. เอาโค้ดหน้าแอปมาไว้ในเครื่อง: `git clone https://github.com/USERNAME/blimblom.git` หรือแตกไฟล์ zip ที่ export จาก AI Studio Build
2. วางไฟล์เสียงตาม A3 ใน `public/audio/`
3. ใน Antigravity: แถบซ้าย **Projects** → **Create New Project** → **New Project** → เลือกโฟลเดอร์โค้ด

B4. ตั้งค่าความปลอดภัยของ Project (สำคัญ)

กดรูปเฟืองข้างชื่อ project → **Project Settings**:

ตั้งค่า	แนะนำ	เหตุผล
Security Preset	ให้ขออนุมัติก่อนรันคำสั่ง terminal	กัน agent รัน `firebase deploy` หรือลบไฟล์เองโดยไม่ได้ตั้งใจ
Agent Behaviour (artifact review)	ให้ขอรีวิวแผนก่อน ไม่ใช่ Always Proceed	ได้อ่านแผนก่อน agent ลงมือแก้หลายไฟล์
Local Permissions	จำกัดให้แก้เฉพาะโฟลเดอร์โปรเจกต์	ไม่ให้แตะไฟล์อื่นในเครื่อง

B5. วิธีทำงานกับ agent

- **หนึ่งงานต่อหนึ่ง conversation** กด **New Conversation** แล้วตั้งชื่อให้ตรงงาน เช่น "C3 alert list"
- งานใหญ่พิมพ์ `/plan` นำหน้า agent จะทำ **Implementation Plan** ให้อ่านก่อน เปิดดูได้จาก **Auxiliary Pane** มุมขวาบน ถ้าโอเคกด **Proceed** ถ้าไม่โอเคพิมพ์ความเห็นลงในแผนได้
- หลังทำเสร็จ อ่าน **Walkthrough** (สรุปสิ่งที่แก้และวิธีทดสอบ) และดู diff ทุกไฟล์ก่อนยอมรับ
- ใช้ `/browser` ให้ agent เปิด Chrome ทดสอบหน้าแอปเอง (ต้องอนุญาตก่อน)
- เริ่มทุก conversation ใหม่ด้วย prompt C0 เพื่อให้ agent รู้ข้อจำกัดของโปรเจกต์
- ทดสอบในเครื่องด้วย `firebase emulators:start --only hosting` หรือ `npx serve public` ก่อน deploy

ส่วน C – Prompt สำเร็จรูป (คัดลอกไปวางได้เลย)

เขียนเป็นภาษาอังกฤษเพราะ agent เข้าใจและเขียนโค้ดได้แม่นกว่า ลำดับการใช้: C0 → C1 → C2 → C3 → C4 → (deploy C6) → C5 → C8 → C9 ส่วน C7 ใช้ทุกครั้งที่เจอบั๊ก

C0 – บริบทโปรเจกต์ (วางเป็นข้อความแรกของทุก conversation)

```
/plan
Project: BlimBlom, a PWA for park rangers in Thailand. Simulated forest audio
sensors send clips to an existing Cloud Run API, Gemini analyzes them, results
are stored in Firestore, and this app shows them live.

Hard constraints:
- Frontend only. The backend already exists: POST {CLOUD_RUN_URL}/analyze.
  Do not create or modify any backend code.
- Request body: { sensor_id, sensor_name, location: {lat, lng}, audio_url,
  mime_type, audio_base64 }
- Response: { id, pushed, event_type, confidence, urgency, reasoning,
  alert_message_en, alert_message_th }
- Firestore: "incidents" is written by the API; the app may only update the
  "status" field. "subscribers/{token}" is written by the app with
  { token, created_at }.
- Plain HTML/CSS/JS, Firebase JS SDK loaded from the official gstatic CDN.
  No build step, no framework.
- Never put a Gemini API key in this code. The app never calls Gemini.
- All UI text in English; alert_message_th is shown as secondary text.
- Must work as an installed PWA on iPhone Safari (iOS 16.4+) and in Chrome.
- Deployed with Firebase Hosting from public/. firebase-messaging-sw.js must
  be at the root of public/.
First read the existing files and summarize the current structure. Do not
change anything until I approve the plan.
```

C1 – ตั้งค่าโค้ดที่ได้จาก AI Studio Build

```
Put all configuration in one file, public/js/config.js, exporting
FIREBASE_CONFIG, VAPID_KEY and CLOUD_RUN_URL, with clearly marked TODO values.
Replace every hard-coded copy elsewhere with imports from config.js.
Remove any code that calls Gemini directly or contains a Gemini API key.
List every file you changed.
```

จากนั้นเปิด `config.js` แล้วใส่ค่าจริงเอง (firebaseConfig และ VAPID key จาก Firebase Console, URL จาก Cloud Run)

C2 – ปุ่มจำลองเซนเซอร์

```
Add a "Simulate sensor event" panel to the home screen.
- Define 4 fake sensors in public/js/sensors.js:
  S01 "Kui Buri North Gate" (12.16, 99.53) -> audio/elephant_1.mp3
  S02 "Ban Rai Farm Edge"   (12.14, 99.55) -> audio/elephant_2.mp3
  S03 "Deep Forest East"    (12.20, 99.58) -> audio/chainsaw_1.wav
  S04 "Stream Bank"         (12.12, 99.50) -> audio/crickets_1.wav
- One button per sensor. On tap: fetch the clip as a Blob, convert to base64
  with FileReader.readAsDataURL and strip the "data:...;base64," prefix.
  Do NOT use String.fromCharCode(...bytes); it crashes on files this size.
- Set mime_type from the extension: .mp3 -> audio/mp3, .wav -> audio/wav.
- POST to CLOUD_RUN_URL + "/analyze" with audio_url set to the clip path.
- While waiting: disable the button and show a spinner (Gemini can take
  several seconds). On success: toast with event_type and urgency.
  On failure: readable error message, button re-enabled.
- Do not write to Firestore in this flow; the API does that.
```

C3 – หน้ารายการแจ้งเตือน

```
Build the alert list on the home screen with a Firestore onSnapshot listener
on "incidents", ordered by created_at descending, limit 50.
Each row: event type label, urgency badge (high = red, medium = amber,
low = gray), sensor_name, relative time ("2 min ago"), and a one-line preview
of alert_message_en. created_at can be null for a moment after a new document
is created (pending server timestamp); show "just now" instead of crashing.
New rows appear without refresh and briefly highlight. Tapping a row opens
the detail view. Show a friendly empty state when there are no incidents.
```

C4 – หน้ารายละเอียดเหตุการณ์

```
Build the incident detail view: sensor_name, coordinates with an
"Open in Google Maps" link, time, an <audio> player using audio_url,
reasoning, confidence as a percentage, alert_message_en and alert_message_th,
and the current status as a badge.
Add three buttons: Acknowledge, On the way, Resolved. Each updates ONLY the
"status" field to acknowledged / en_route / resolved (the Firestore rules
reject changes to any other field). Show an error if the update fails.
Support deep links: if the page loads with ?incident=<id>, open that
incident directly. A back button returns to the list.
```

C5 – ติดตั้งบน iPhone + push notification

```
Implement install and push support:
1. public/manifest.json: name "BlimBlom", display "standalone", start_url "/",
   theme color, icons 192x192 and 512x512 (generate simple placeholder PNGs).
   Add apple-touch-icon and apple-mobile-web-app-capable meta tags.
2. If not running standalone (navigator.standalone or
   matchMedia('(display-mode: standalone)')), show a dismissible banner:
   on iOS "Tap Share, then Add to Home Screen"; elsewhere use the
   beforeinstallprompt event for an Install button.
3. In standalone mode show an "Enable alerts" button. Only when tapped:
   Notification.requestPermission(), register /firebase-messaging-sw.js,
   getToken(messaging, { vapidKey: VAPID_KEY, serviceWorkerRegistration }),
   then setDoc(doc(db, 'subscribers', token),
   { token, created_at: serverTimestamp() }). Never ask on page load.
4. public/firebase-messaging-sw.js: initialize Firebase with the same config
   (compat SDK via importScripts is fine). The server sends messages with a
   "notification" block, which the browser displays itself, so do not show a
   second notification in onBackgroundMessage. On notificationclick, open or
   focus /?incident=<data.incident_id>.
5. Show a foreground in-app toast when a message arrives while the app is open.
Afterwards, tell me step by step how to test this on an iPhone.
```

C6 – Deploy ขึ้น Firebase Hosting

```
Check that firebase.json serves the public/ folder. Before deploying, list
every file that will be uploaded and confirm none contains a secret (the
firebaseConfig values and VAPID public key are fine; a Gemini key is not).
Then run: firebase deploy --only hosting
and give me the Hosting URL.
```

agent จะขออนุมัติก่อนรันคำสั่ง (เพราะตั้ง Security Preset ไว้ใน B4) อ่านรายการไฟล์ก่อนกดอนุมัติ

C7 – แก้บั๊ก (ใช้ทุกครั้งที่เจอปัญหา)

```
Bug: <what I did> -> <what happened> -> <what I expected>
Error (console or terminal): <paste the exact error>
Device / browser: <e.g. iPhone iOS 17, installed PWA / Chrome on Windows>
Find the root cause before changing any code. Explain the cause in 2-3
sentences, then make the smallest fix. Do not refactor unrelated code.
```

ถ้า error มาจาก Cloud Run (เช่น 500) ให้ดู log ใน Cloud Shell ด้วย `gcloud run services logs read blimblom-api --region asia-southeast1 --limit 50` แล้ววาง log ลงใน prompt

C8 – ขัดเกลาสำหรับถ่ายวิดีโอเดโม

```
Polish the UI for a 3-minute demo video without changing behavior:
larger tap targets, consistent spacing, urgency colors with strong contrast,
a header "BlimBlom · Forest Alert", and a subtle animation when a new
incident arrives. Check the layout at 375px width with /browser and fix any
overflow or horizontal scrolling. Show me before/after screenshots.
```

C9 – ตรวจครั้งสุดท้ายก่อนส่งงาน

```
Final review. Read-only first, do not change anything yet:
1. Any secrets in the code or git history (besides firebaseConfig and the
   VAPID public key)?
2. Is all UI text in English?
3. Any console errors on first load?
4. Walk the full flow with /browser: simulate -> list updates -> open detail
   -> change status.
Report findings as a numbered list. Fix only the items I approve.
```

กฎสั้น ๆ เวลาสั่งงาน agent

- **ห้ามให้ agent แก้ backend หรือเปลี่ยนรูปแบบข้อมูลที่ส่งไป Cloud Run** ถ้ามันเสนอ ให้ปฏิเสธ (C0 กันไว้แล้ว)
- สั่งทีละเรื่อง เล็กและชัด ดีกว่าสั่งรวมหลายฟีเจอร์ใน prompt เดียว
- อ่าน diff ก่อนยอมรับทุกครั้ง โดยเฉพาะไฟล์ config และ service worker
- Commit ขึ้น GitHub หลังแต่ละงานที่ทำงานได้ จะได้ย้อนกลับได้ถ้า agent ทำพัง
- หลังหยุดเพิ่มฟีเจอร์ (คืนวันเสาร์) ใช้แค่ C7 และ C9 เท่านั้น